

Climbing up a ladder: a new approach to contextual ~~refinement~~ improvement

Koko Muroya
(NII, Japan)

Special thanks to...

Dan R. Ghica (Univ. Birmingham & Huawei Central Software Institute)

Todd Waugh Ambridge (Univ. Birmingham), Makoto Hamana (Kyusyu Institute of Technology),

Natsuki Urabe (NII), Takahiro Sanada (Fukui Prefectural Univ.)

My research area

Modelling program execution

- software science
 - trustable software (outcome & efficiency)
 - verification of systems that execute programs

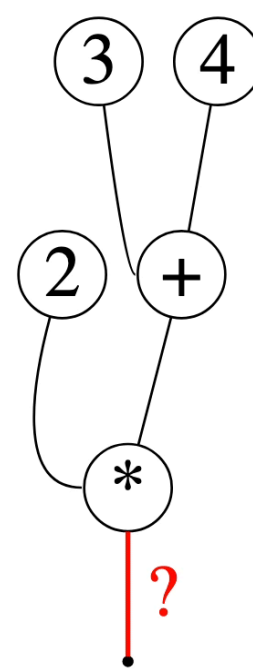
Modelling program execution

- software science
 - trustable software (outcome & efficiency)
 - verification of systems that execute programs

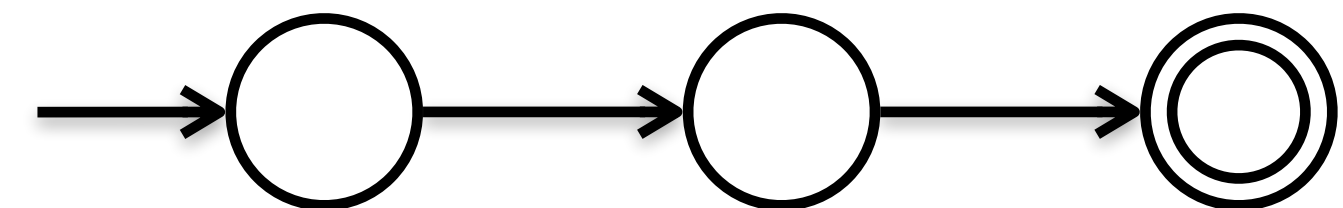
term rewriting system

$$2 \times (3 + 4) \rightarrow 2 \times 7 \rightarrow 14$$

graph rewriting system



(nondeterministic) automaton



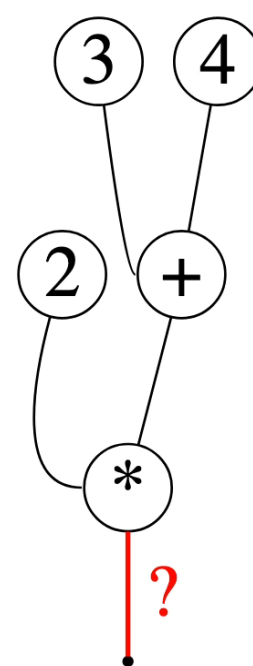
Modelling program execution

- software science
 - trustable software (outcome & efficiency)
 - verification of systems that execute programs

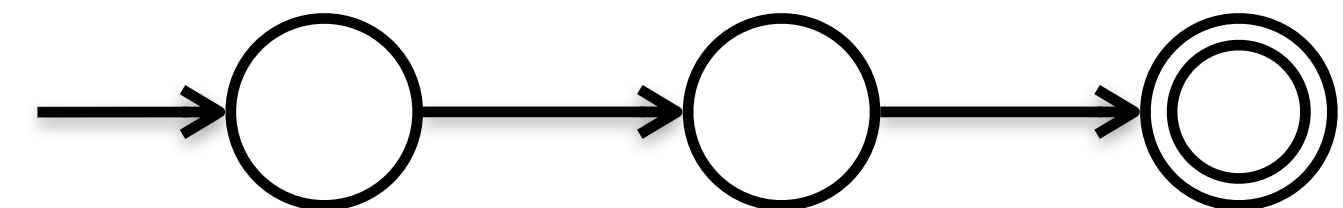
term rewriting system

$2 \times (3 + 4) \rightarrow 2 \times 7 \rightarrow 14$

graph rewriting system



(nondeterministic) automaton



Contextual improvement between programs [Sands '96]

- software science
 - trustable software (outcome & efficiency)
 - verification of systems that execute programs
 - mathematical proof of *contextual improvement* between programs

Contextual improvement between programs [Sands '96]

- “The (observable) result of t can be reproduced by u , *more efficiently*.”
- proved *contextual improvement* $\Rightarrow$ certified compiler optimisation
- formally, for program fragments t and u ,

$$t \succeq_{\text{ctxt}} u \stackrel{\Delta}{\iff} \forall C, v. \left(\mathcal{S}_{C[t]} \models \downarrow^k \{v\} \implies (\mathcal{S}_{C[u]} \models \downarrow^m \{v\} \wedge k \geq m) \right)$$

system
specialised to
program $C[t]$

termination
with result v
in k steps

system
specialised to
program $C[u]$

termination
with result v
in m steps

- cf. model checking $\mathcal{S} \models \phi$

Contextual improvement via “simulation”

1. define a “simulation” notion (for automata)

- to assert that s is imitated by s' with a less number of steps

2. fix a system $\mathcal{S}$ (i.e. execution model of programs)

- key: explicit notion of step to define $\mathcal{S} \models \underline{\downarrow^k \{v\}}$

termination
with result v
in k steps

3. prove a “simulation”

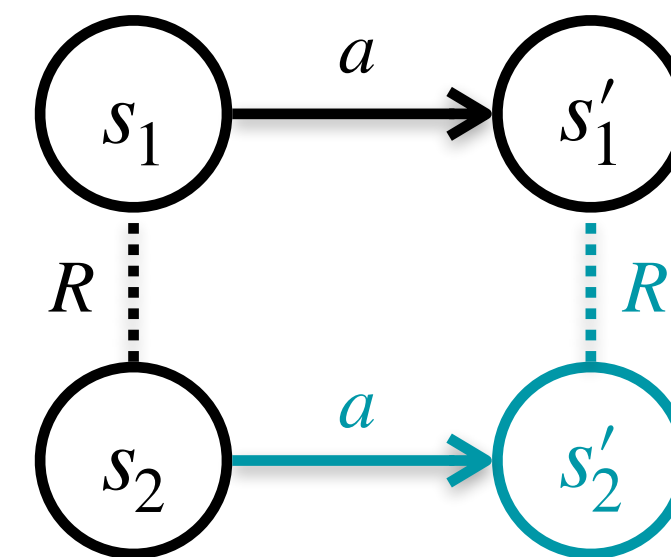
- $\underline{\mathcal{S}_{C[t]}}$ is “simulated” by $\underline{\mathcal{S}_{C[u]}}$ with a less number of steps

system
specialised to
program $C[t]$

system
specialised to
program $C[u]$

Standard notion of *simulation*

- to assert that s is imitated by s' in a **stepwise manner**
- simulation $R \subseteq S_1 \times S_2 \iff \forall (s_1, s_2) \in R. \forall (s_1 \xrightarrow{a} s'_1). \exists s'_2.$

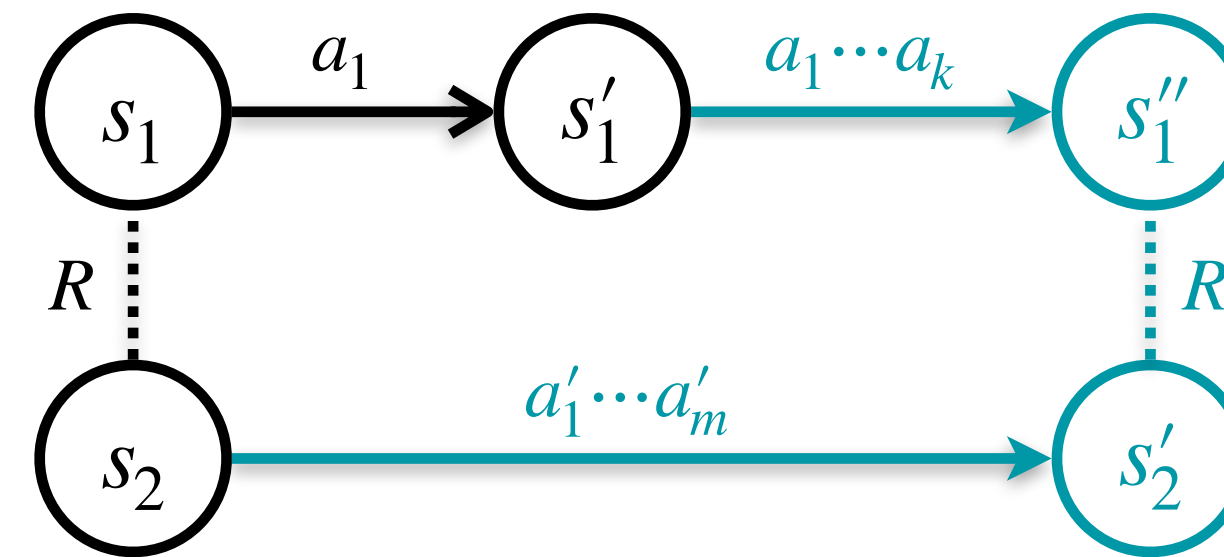


- **Prop.** (soundness) language inclusion $\mathcal{L}(s_1) \subseteq \mathcal{L}(s_2)$ holds
if $\exists R$: simulation such that $(s_1, s_2) \in R$

Contextual improvement via “simulation”

- to assert that s is “simulated” by s' in a **quasi-stepwise manner**

- “simulation” $R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R. \forall (s_1 \xrightarrow{a} s'_1). \exists s''_1, s'_2. \exists (k + 1 \geq m).$



- **Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

if $\exists R$: “simulation” such that $\forall C. (C[t], C[u]) \in R$

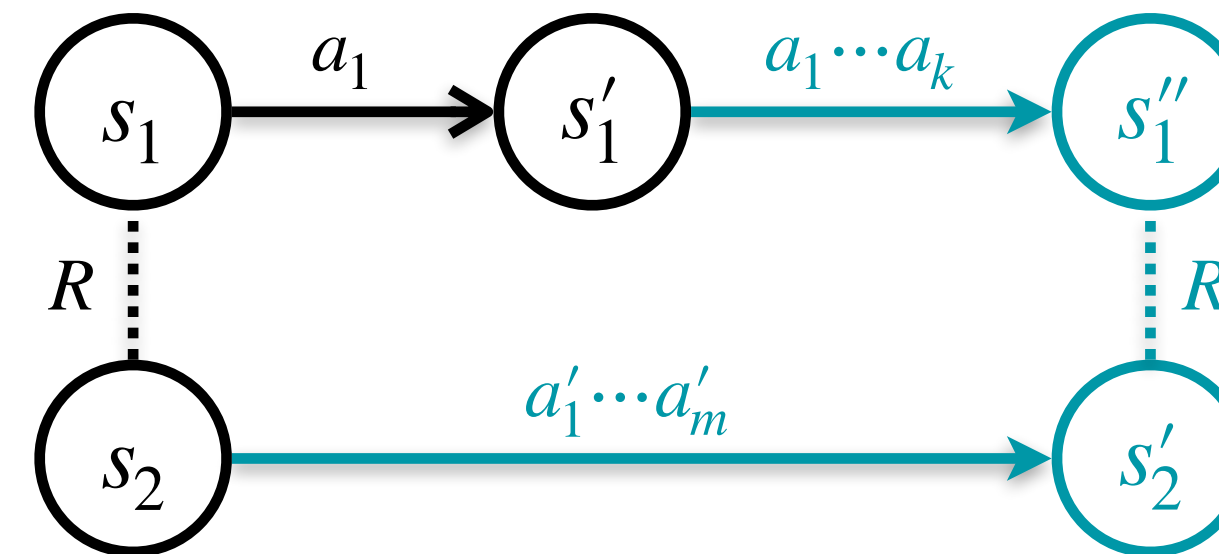
Contextual improvement via “simulation”

- to assert that s is “simulated” by s' in a **quasi-stepwise manner**

- “simulation” $R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R. \forall (s_1 \xrightarrow{a} s'_1). \exists s''_1, s'_2. \exists (k + 1 \geq m).$

1. define a “simulation”

2. fix a system
(i.e. execution model)



3. prove a “simulation”

- **Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

if $\exists R$: “simulation” such that $\forall C. (C[t], C[u]) \in R$

Contextual improvement via “simulation”

The first attempt

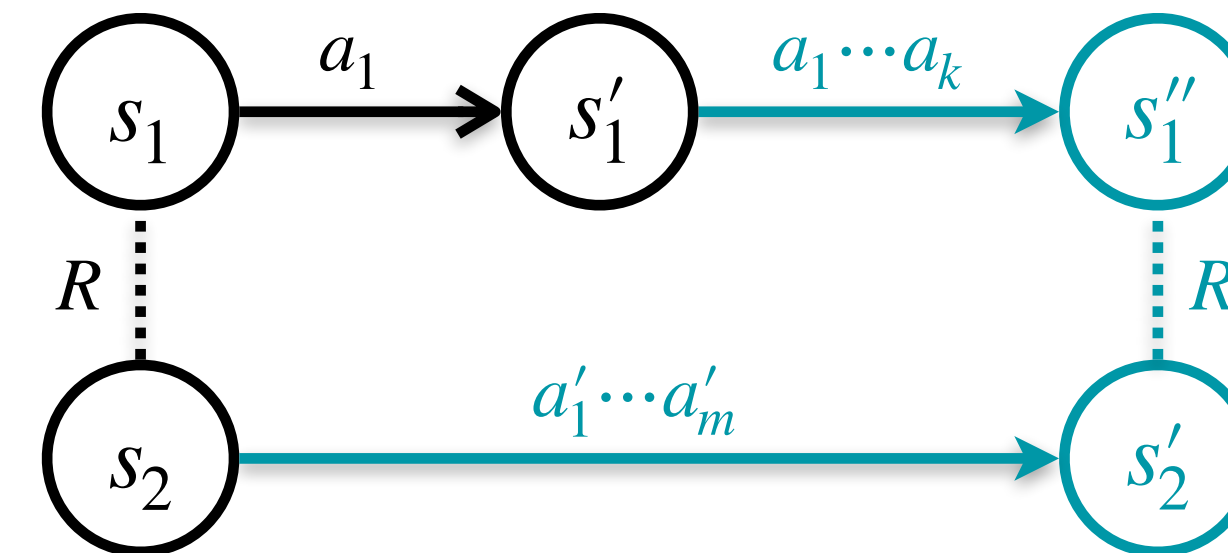
	[M., PhD thesis '20] [M.+ , arxiv '19]
1. define a “simulation”	<i>counting simulation</i>
2. fix a system	graph rewriting systems
3. prove a “simulation”	😞 elementary but tedious case analysis

The first attempt

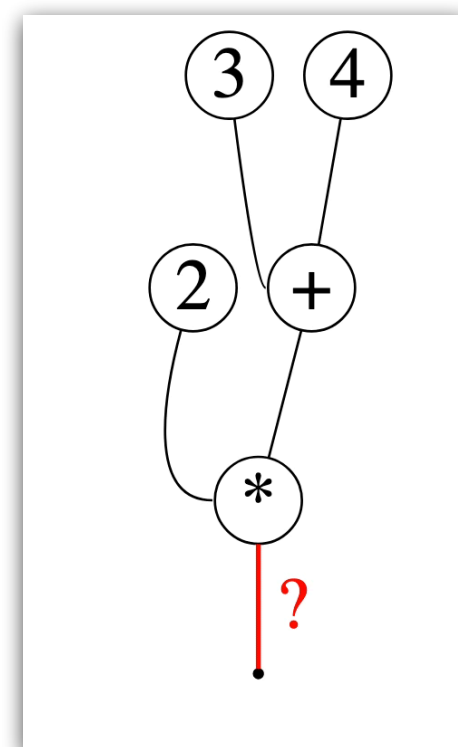
Counting simulation with graph rewriting systems

- counting simulation

$$R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R. \forall (s_1 \xrightarrow{a} s'_1). \exists s''_1, s'_2. \exists (k+1 \geq m).$$



- graph rewriting systems



- Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

if $\exists R$: counting simulation such that $\forall C. (C[t], C[u]) \in R$

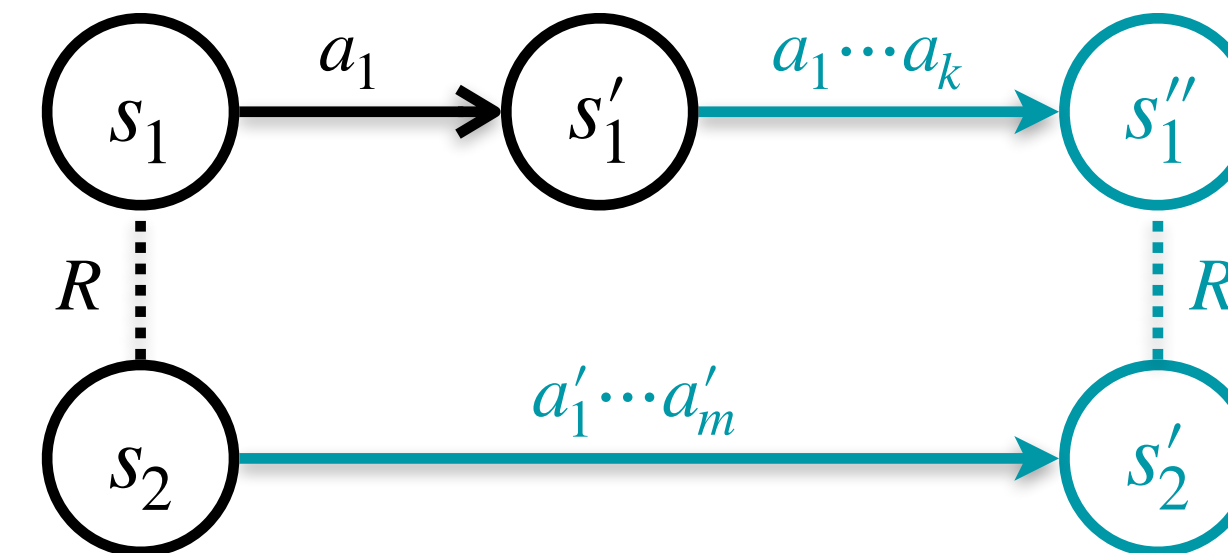
elementary but error-prone case analysis

The first attempt

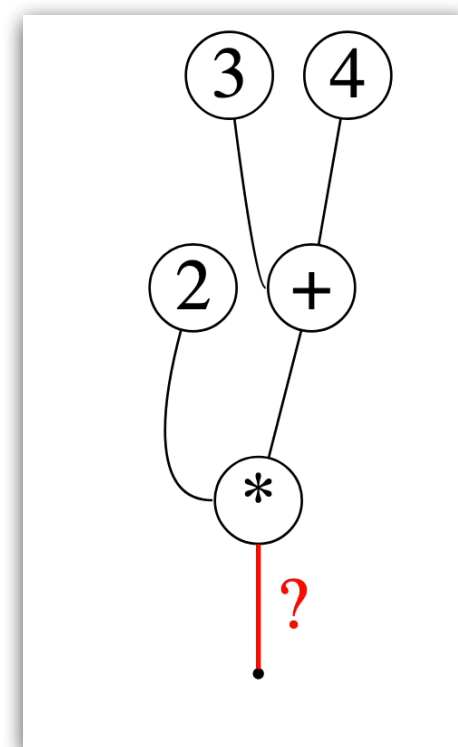
Counting simulation with graph rewriting systems

- counting simulation

$$R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R. \forall (s_1 \xrightarrow{a} s'_1). \exists s''_1, s'_2. \exists (k+1 \geq m).$$



- graph rewriting systems



- Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

if $\exists R$: counting simulation such that $\forall C. (C[t], C[u]) \in R$

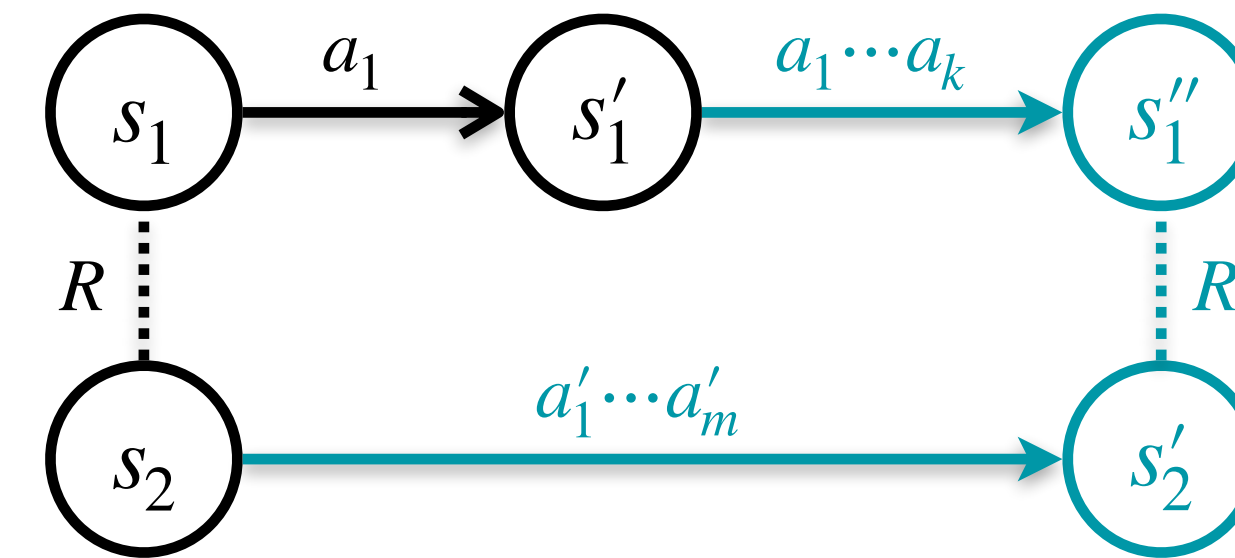
elementary but error-prone case analysis

Refining the approach

	[M., PhD thesis '20] [M.+, arxiv '19]	[M.+, FLOPS '24]
1. define a “simulation”	<i>counting simulation</i>	<i>local coherence</i> [Huet '80]
2. fix a system	graph rewriting systems	term rewriting systems <i>term evaluation systems</i>
3. prove a “simulation”	😞 elementary but tedious case analysis	😊 automatable <i>critical pair analysis</i>

Local coherence with term rewriting evaluation systems

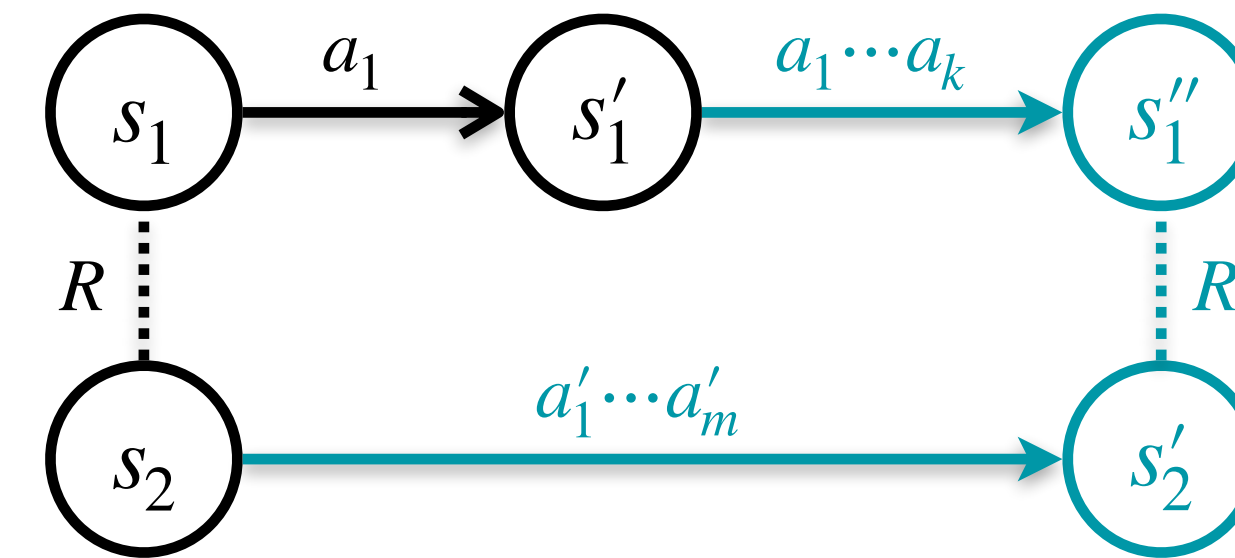
- *counting simulation* $R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R . \forall s_1 \xrightarrow{a} s'_1 . \exists s''_1, s'_2 .$



- **aka. local coherence** [Huet '80]
- term rewriting systems
- Prop. (soundness) *contextual improvement* $t \succeq_{ctx} u$ holds
if $\exists R$: *counting simulation* such that $\forall C . (C[t], C[u]) \in R$

Local coherence with term rewriting evaluation systems

- counting simulation $R \subseteq S_1 \times S_2 \xLeftrightarrow{\Delta} \forall (s_1, s_2) \in R . \forall s_1 \xrightarrow{a} s'_1 . \exists s''_1, s'_2 .$



- aka. **local coherence** [Huet '80]
- ~~term rewriting systems~~

term evaluation systems

$$\frac{(l \rightarrow r) \in \mathcal{E} \quad E \in Ectx}{E[l\theta] \rightarrow_{\mathcal{E}} E[r\theta]}$$

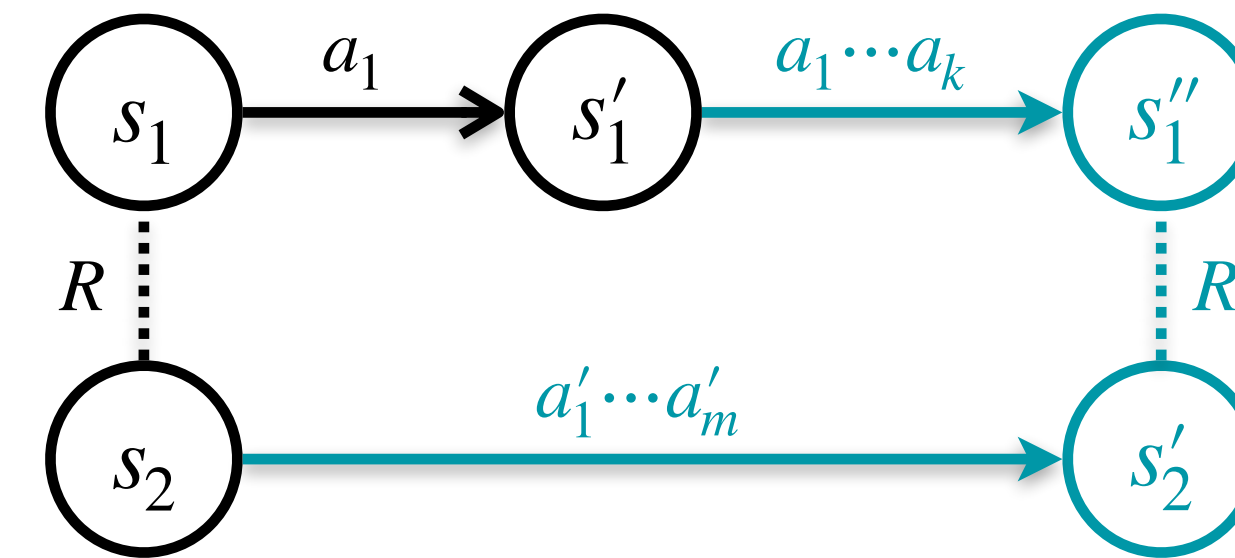
$$\frac{(l \Rightarrow r) \in \mathcal{R} \quad C \in Ctx}{C[l\theta] \Rightarrow_{\mathcal{R}} C[r\theta]}$$

- Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

if $\exists R$: counting simulation such that $\forall C . (C[t], C[u]) \in R$

Local coherence with term rewriting evaluation systems

- counting simulation $R \subseteq S_1 \times S_2 \xLeftrightarrow{\Delta} \forall (s_1, s_2) \in R . \forall s_1 \xrightarrow{a} s'_1 . \exists s''_1, s'_2 .$



- aka. **local coherence** [Huet '80]
- ~~term rewriting systems~~

term evaluation systems

$$\frac{(l \rightarrow r) \in \mathcal{E} \quad E \in Ectx}{E[l\theta] \rightarrow_{\mathcal{E}} E[r\theta]}$$

$$\frac{(l \Rightarrow r) \in \mathcal{R} \quad C \in Ctx}{C[l\theta] \Rightarrow_{\mathcal{R}} C[r\theta]}$$

- Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds (for determinism)

if every critical pair between $\rightarrow_{\mathcal{E}} , \Rightarrow_{\mathcal{R}}$ is joinable

Local coherence with term rewriting evaluation systems

- An example: λ -calculus with effect handlers [Pretner '15]

Syntax class *Sclass*

functions $F ::= x \mid \text{fun}(x.P)$

values $V ::= \text{true} \mid \text{false} \mid F \mid H$

handlers $H ::= \text{handler}_1(x.P, x.k.P_1) \mid \text{handler}_0(x.P)$

computations $P, P_1, P_2 ::= \text{return}(V) \mid \text{op}(V, y.P) \mid \text{do}(P_1, x.P_2) \mid \text{if}(V, P_1, P_2) \mid F V \mid \text{with_handle}(H, P)$

Evaluation contexts *Ectx* $E ::= \square \mid \text{do}(E, x.P) \mid \text{with_handle}(H, E)$

Evaluation rules $\mathcal{E}$ where $i \in [2]$

$\text{do}(\text{return}(V), x.P[x]) \rightarrow P[V]$ (1)

$\text{do}(\text{op}_i(V, y.P_1[y]), x.P_2[x]) \rightarrow \text{op}_i(V, y.\text{do}(P_1[y], x.P_2[x]))$ (2)

$\text{if}(\text{true}, P_1, P_2) \rightarrow P_1$ (3)

$\text{if}(\text{false}, P_1, P_2) \rightarrow P_2$ (4)

$\text{fun}(x.P[x]) V \rightarrow P[V]$ (5)

In the following three rules, $h_1 \equiv \text{handler}_1(x.P[x], x.k.P_1[x, k])$.

$\text{with_handle}(h_1, \text{return}(V)) \rightarrow P[V]$ (6)

$\text{with_handle}(h_1, \text{op}_1(V, y.P'[y])) \rightarrow P_1[V, \text{fun}(y.P'[y])]$ (7)

$\text{with_handle}(h_1, \text{op}_1(V, y.P'[y])) \rightarrow P_1[V, \text{fun}(y.\text{with_handle}(h_1, P'[y]))]$ (7')

$\text{with_handle}(h_1, \text{op}_2(V, y.P'[y])) \rightarrow \text{op}_2(V, y.\text{with_handle}(h_1, P'[y]))$ (8)

In the following two rules, $h_0 \equiv \text{handler}_0(x.P[x])$.

$\text{with_handle}(h_0, \text{return}(V)) \rightarrow P[V]$ (9)

$\text{with_handle}(h_0, \text{op}_i(V, y.P'[y])) \rightarrow \text{op}_i(V, y.\text{with_handle}(h_0, P'[y]))$ (10)

Refinement rules $\mathcal{R}$

$\text{do}(P, x.\text{return}(x)) \Rightarrow P$ (r3)

$\text{do}(\text{do}(P_1, x_1.P_2[x_1]), x_2.P_3[x_2]) \Rightarrow \text{do}(P_1, x_1.\text{do}(P_2[x_1], x_2.P_3[x_2]))$ (r4)

$\text{if}(V, P[\text{true}], P[\text{false}]) \Rightarrow P[V]$ (r7)

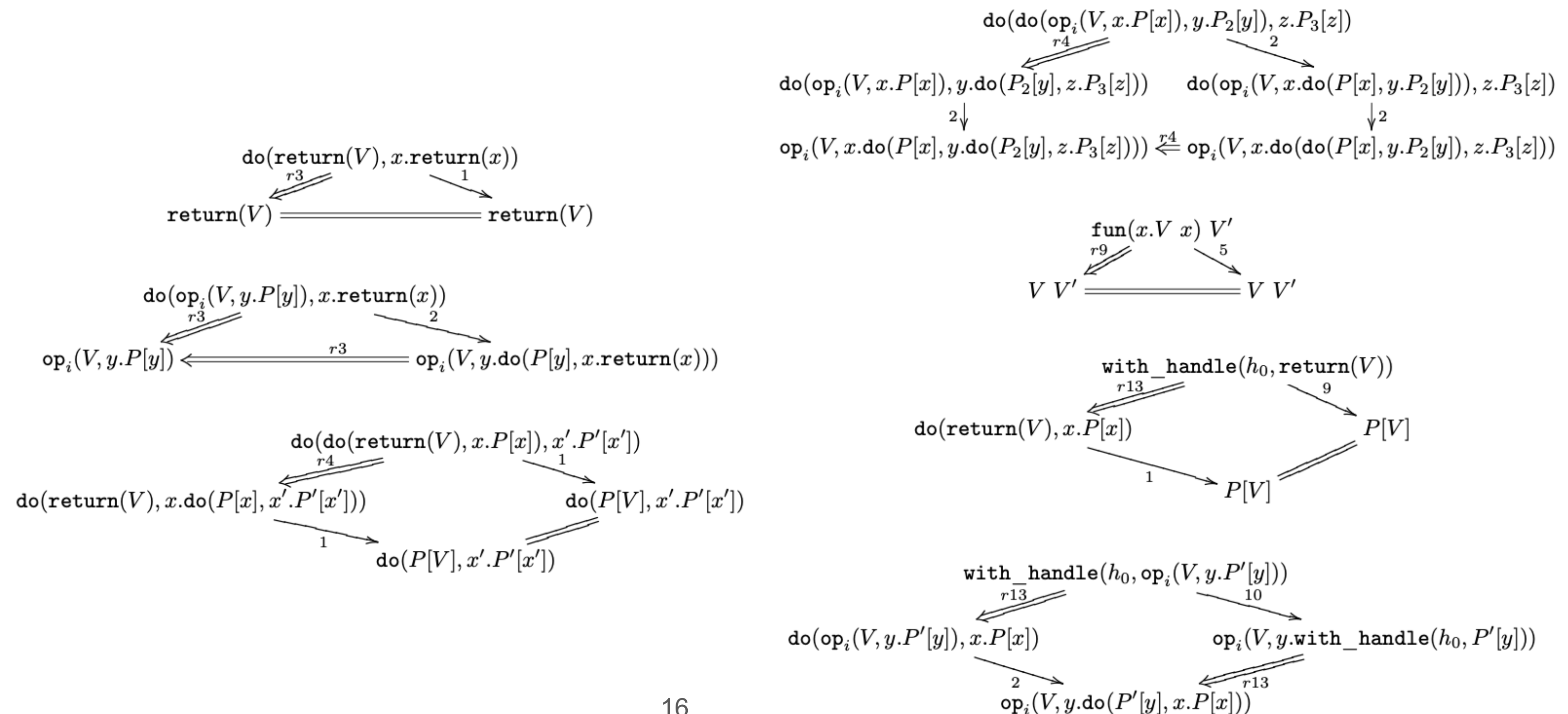
$\text{fun}(x.F x) \Rightarrow F$ (r9)

$\text{with_handle}(\text{handler}_0(x.P[x]), P') \Rightarrow \text{do}(P', x.P[x])$ (r13)

Local coherence with term rewriting evaluation systems

- An example: λ -calculus with effect handlers [Pretner '15]
- **Prop. (soundness)** *contextual improvement* $t \succeq_{ctx} u$ holds (for determinism)

if every *critical pair* between $\rightarrow_{\mathcal{E}}$, $\Rightarrow_{\mathcal{R}}$ is *joinable*



Local coherence with term rewriting evaluation systems

- An example: λ -calculus with effect handlers [Pretner '15]
- Prop. (soundness) *contextual improvement* $t \succeq_{ctx} u$ holds (for determinism)

if every *critical pair* between $\rightarrow_{\mathcal{E}}$, $\Rightarrow_{\mathcal{R}}$ is joinable

B.2 Local Coherence Check

*Ex> lcoh evals refis

```
1: Overlap (1-v)-(r3-v)--- P'|-> return(V), P|-> z1.return(z1) -----
(1-v) |do(return(V),x.P[x])| => P[V]
(r3-v) do(P',x'.return(x')) => P'
      do(return(V),x.return(x))
return(V) <-(1-v)-^-(r3-v)-> return(V)
----> return(V) =OK= return(V) <---
```

```
2: Overlap (r4-v)-(1-v)--- P1|-> return(V'), P'|-> z1.P2[z1] -----
(r4-v) do(|do(P1,x.P2[x])|,x.P3[x]) => do(P1,x1.do(P2[x1],x2.P3[x2]))
(1-v) do(return(V'),x'.P'[x']) => P'[V']
      do(do(return(V'),x.P2[x]),x.P3[x])
do(return(V'),x17.do(P2[x17],x27.P3[x27])) <-(r4-v)-^-(1-v)-> do(P2[V'],x.P3[x])
----> do(P2[V'],x27.P3[x27]) =OK= do(P2[V'],x.P3[x]) <---
```

```
3: Overlap (2-1-v)-(r3-v)--- P'|-> op1(V,y.P1[y]), P2|-> z1.return(z1) -----
(2-1-v) |do(op1(V,y.P1[y]),x.P2[x])| => op1(V,y.do(P1[y],x.P2[x]))
(r3-v) do(P',x'.return(x')) => P'
      do(op1(V,y.P1[y]),x.return(x))
op1(V,y14.do(P1[y14],x14.return(x14))) <-(2-1-v)-^-(r3-v)-> op1(V,y.P1[y])
----> op1(V,y14.do(P1[y14],x14.return(x14))) =OK= op1(V,y.P1[y]) <---
```

```
4: Overlap (r4-v)-(2-1-v)--- P1|-> op1(V',y'.P1'[y']), P2'|-> z1.P2[z1] -----
(r4-v) do(|do(P1,x.P2[x])|,x.P3[x]) => do(P1,x1.do(P2[x1],x2.P3[x2]))
(2-1-v) do(op1(V',y'.P1'[y']),x'.P2'[x']) => op1(V',y'.do(P1'[y'],x'.P2'[x']))
      do(do(op1(V',y'.P1'[y']),x.P2[x]),x.P3[x])
do(op1(V',y'.P1'[y']),x121.do(P2[x121],x1.P3[x1])) <-(r4-v)-^-(2-1-v)->
do(op1(V',yd.do(P1'[yd],xd.P2[xd])),x.P3[x])
----> op1(V',y23.do(P1'[y23],x23.do(P2[x23],x1.P3[x1])) =OK=
op1(V',y26.do(do(P1'[y26],xd.P2[xd]),x26.P3[x26])) <---
```

```
5: Overlap (2-2-v)-(r3-v)--- P'|-> op2(V,y.P1[y]), P2|-> z1.return(z1) -----
(2-2-v) |do(op2(V,y.P1[y]),x.P2[x])| => op2(V,y.do(P1[y],x.P2[x]))
(r3-v) do(P',x'.return(x')) => P'
      do(op2(V,y.P1[y]),x.return(x))
op2(V,y33.do(P1[y33],x33.return(x33))) <-(2-2-v)-^-(r3-v)-> op2(V,y.P1[y])
----> op2(V,y33.do(P1[y33],x33.return(x33))) =OK= op2(V,y.P1[y]) <---
```

```
6: Overlap (r4-v)-(2-2-v)--- P1|-> op2(V',y'.P1'[y']), P2'|-> z1.P2[z1] -----
(r4-v) do(|do(P1,x.P2[x])|,x.P3[x]) => do(P1,x1.do(P2[x1],x2.P3[x2]))
(2-2-v) do(op2(V',y'.P1'[y']),x'.P2'[x']) => op2(V',y'.do(P1'[y'],x'.P2'[x']))
```

```
do(do(op2(V',y'.P1'[y']),x.P2[x]),x.P3[x])
do(op2(V',y'.P1'[y']),x140.do(P2[x140],x240.P3[x240])) <-(r4-v)-^-(2-2-v)->
do(op2(V',yd.do(P1'[yd],xd.P2[xd])),x.P3[x])
----> op2(V',y42.do(P1'[y42],x42.do(P2[x42],x240.P3[x240])) =OK=
op2(V',y45.do(do(P1'[y45],xd.P2[xd]),x45.P3[x45])) <---
```

```
7: Overlap (5-v)-(r9-v)--- P|-> z1.app(V',z1) -----
(5-v) |fun(x.P[x])|@V => P[V]
(r9-v) fun(x'.app(V',x')) => V'
      app(fun(x.app(V',x)),V)
app(V',V) <-(5-v)-^-(r9-v)-> app(V',V)
----> app(V',V) =OK= app(V',V) <---
```

```
8: Overlap (9-v)-(r13-v)--- P'|-> z1.P[z1], Pd'|-> return(V) -----
(9-v) |with(handler0(x.P[x]),return(V))| => P[V]
(r13-v) with(handler0(x'.P'[x']),Pd') => do(Pd',x'.P'[x'])
      with(handler0(x.P[x]),return(V))
P[V] <-(9-v)-^-(r13-v)-> do(return(V),xd66.P[xd66])
----> P[V] =OK= P[V] <---
```

```
9: Overlap (10-1-v)-(r13-v)--- P'|-> z1.P[z1], Pd'|-> op1(V,y.Pd[y]) -----
(10-1-v) |with(handler0(x.P[x]),op1(V,y.Pd[y]))| => op1(V,y.with(handler0(x.P[x]),Pd[y]))
(r13-v) with(handler0(x'.P'[x']),Pd') => do(Pd',x'.P'[x'])
      with(handler0(x.P[x]),op1(V,y.Pd[y]))
op1(V,y72.with(handler0(x72.P[x72]),Pd[y72])) <-(10-1-v)-^-(r13-v)->
do(op1(V,y.Pd[y]),xd73.P[xd73])
----> op1(V,y72.with(handler0(x72.P[x72]),Pd[y72])) =OK=
op1(V,y76.do(Pd[y76],x76.P[x76])) <---
```

```
10: Overlap (10-2-v)-(r13-v)--- P'|-> z1.P[z1], Pd'|-> op2(V,y.Pd[y]) -----
(10-2-v) |with(handler0(x.P[x]),op2(V,y.Pd[y]))| => op2(V,y.with(handler0(x.P[x]),Pd[y]))
(r13-v) with(handler0(x'.P'[x']),Pd') => do(Pd',x'.P'[x'])
      with(handler0(x.P[x]),op2(V,y.Pd[y]))
op2(V,y86.with(handler0(x86.P[x86]),Pd[y86])) <-(10-2-v)-^-(r13-v)->
do(op2(V,y.Pd[y]),xd87.P[xd87])
----> op2(V,y86.with(handler0(x86.P[x86]),Pd[y86])) =OK=
op2(V,y90.do(Pd[y90],x90.P[x90])) <---
```

#Joinable! (Total 10 CPs)
YES

Extending the approach

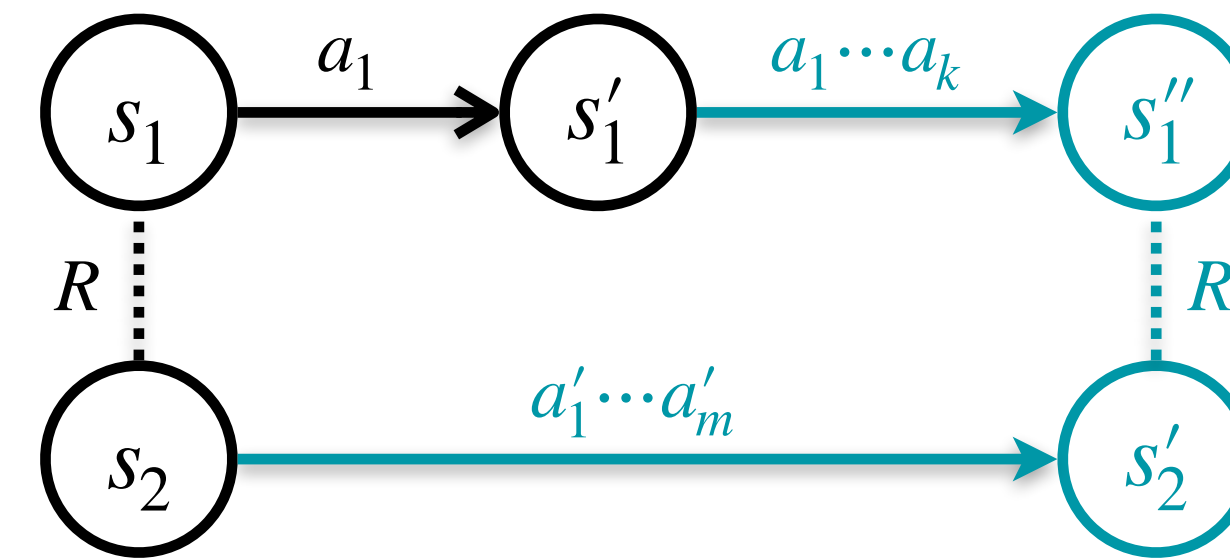
	[M., PhD thesis '20] [M.+ , arxiv '19]	[M.+ , FLOPS '24]
1. define a “simulation”	<i>counting simulation</i>	<i>local coherence</i> [Huet '80]
2. fix a system	graph rewriting systems	term rewriting systems <i>term evaluation systems</i>
3. prove a “simulation”	😞 elementary but tedious case analysis	😊 automatable <i>critical pair analysis</i>

Extending the approach

	[M., PhD thesis '20] [M.+, arxiv '19]	[M.+, FLOPS '24]	[M.+, CMCS '24]
1. define a “simulation”	<i>counting simulation</i>	<i>local coherence</i> [Huet '80]	<i>preorder-constrained simulation</i>
2. fix a system	graph rewriting systems	term rewriting systems <i>term evaluation systems</i>	nondeterministic automata
3. prove a “simulation”	😞 elementary but tedious case analysis	😊 automatable <i>critical pair analysis</i>	(😞 game solving)
divergence	✓	✓	✓
continuation	(✓)	✓	(✓)
mutable state	✓	✗	(✓)
I/O	✗	✗	✓
angelic nondeterminism	✗	✗	✓
demonic nondeterminism	✗	✗	✗
probability	✗	✗	✗

Preorder-constrained simulation (PC-simulation)

- counting simulation $R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R. \forall s_1 \xrightarrow{a} s'_1. \exists s''_1, s'_2.$



- nondeterministic automata

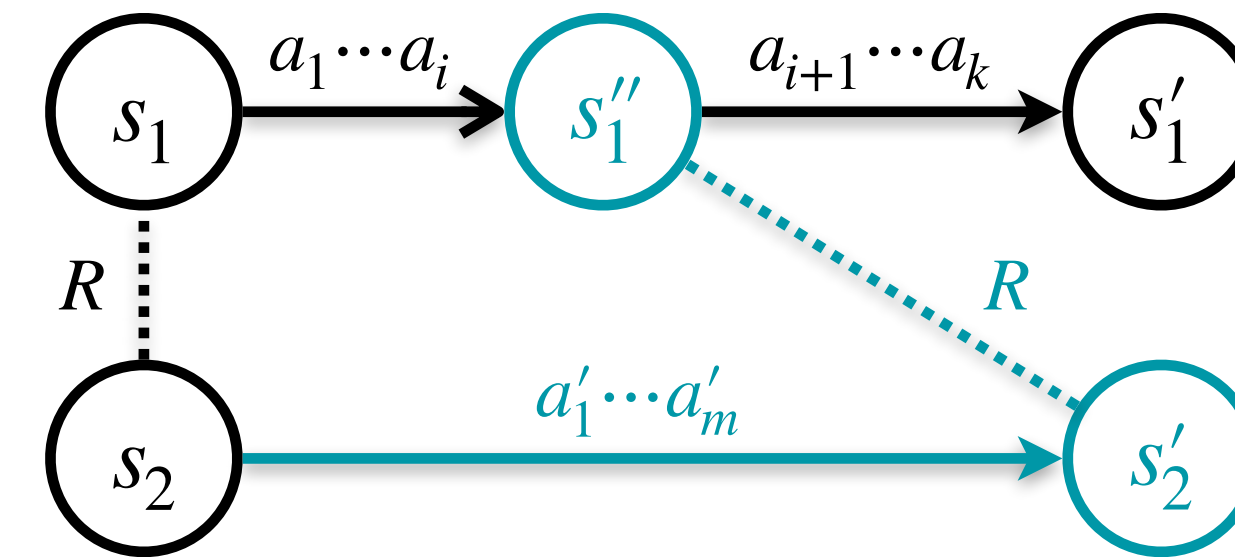
- Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

if $\exists R: PC\text{-simulation}$ such that $\forall C. (C[t], C[u]) \in R$

Preorder-constrained simulation (PC-simulation)

- *PC-simulation*

$$R \subseteq S_1 \times S_2 \stackrel{\Delta}{\iff} \forall (s_1, s_2) \in R. \forall (s_1 \xrightarrow{a_1 \cdots a_k} s'_1). \exists i, s''_1, s'_2. \exists (i \geq m).$$



- nondeterministic automata

- **Prop. (soundness)** contextual improvement $t \succeq_{ctx} u$ holds

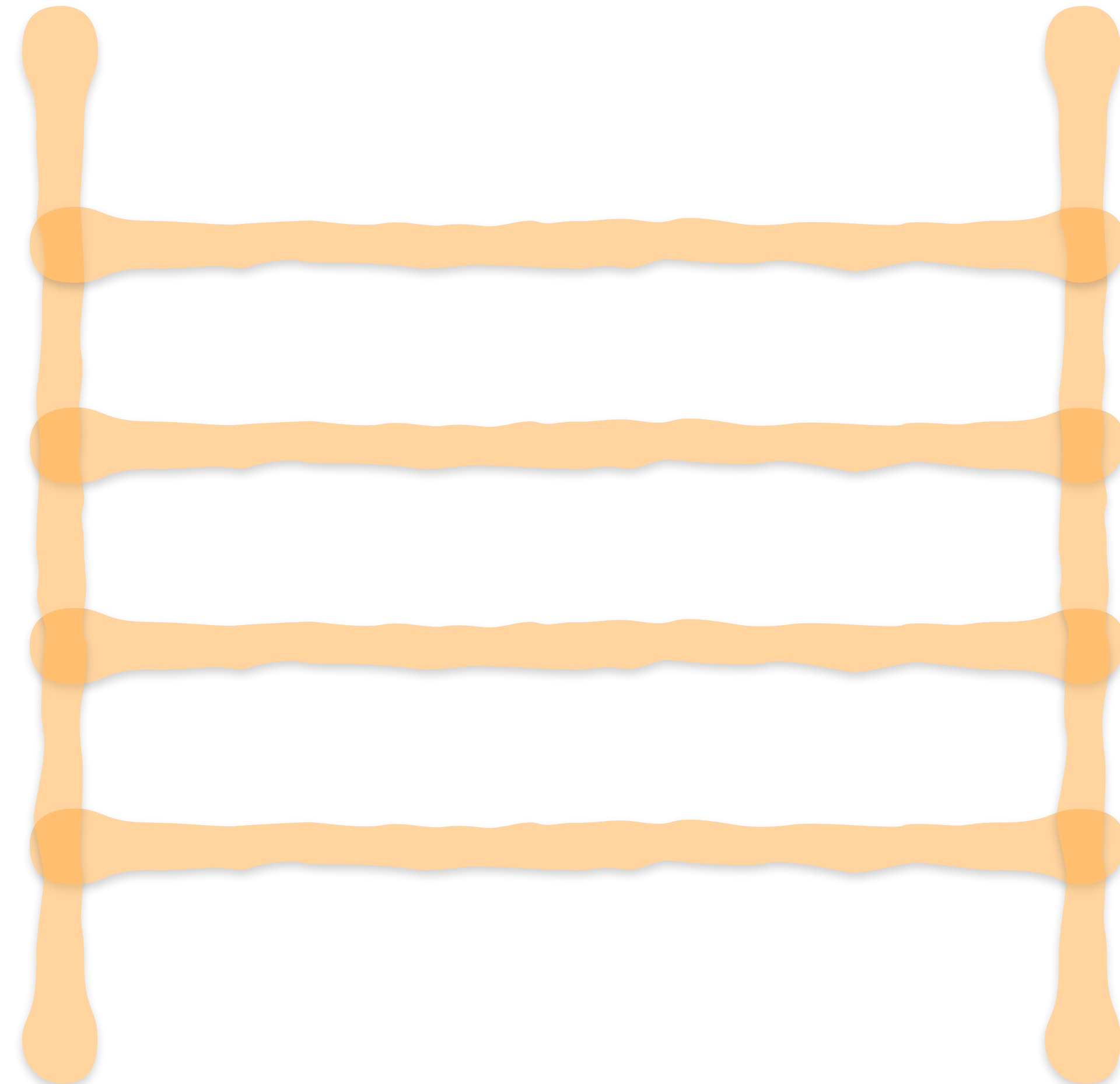
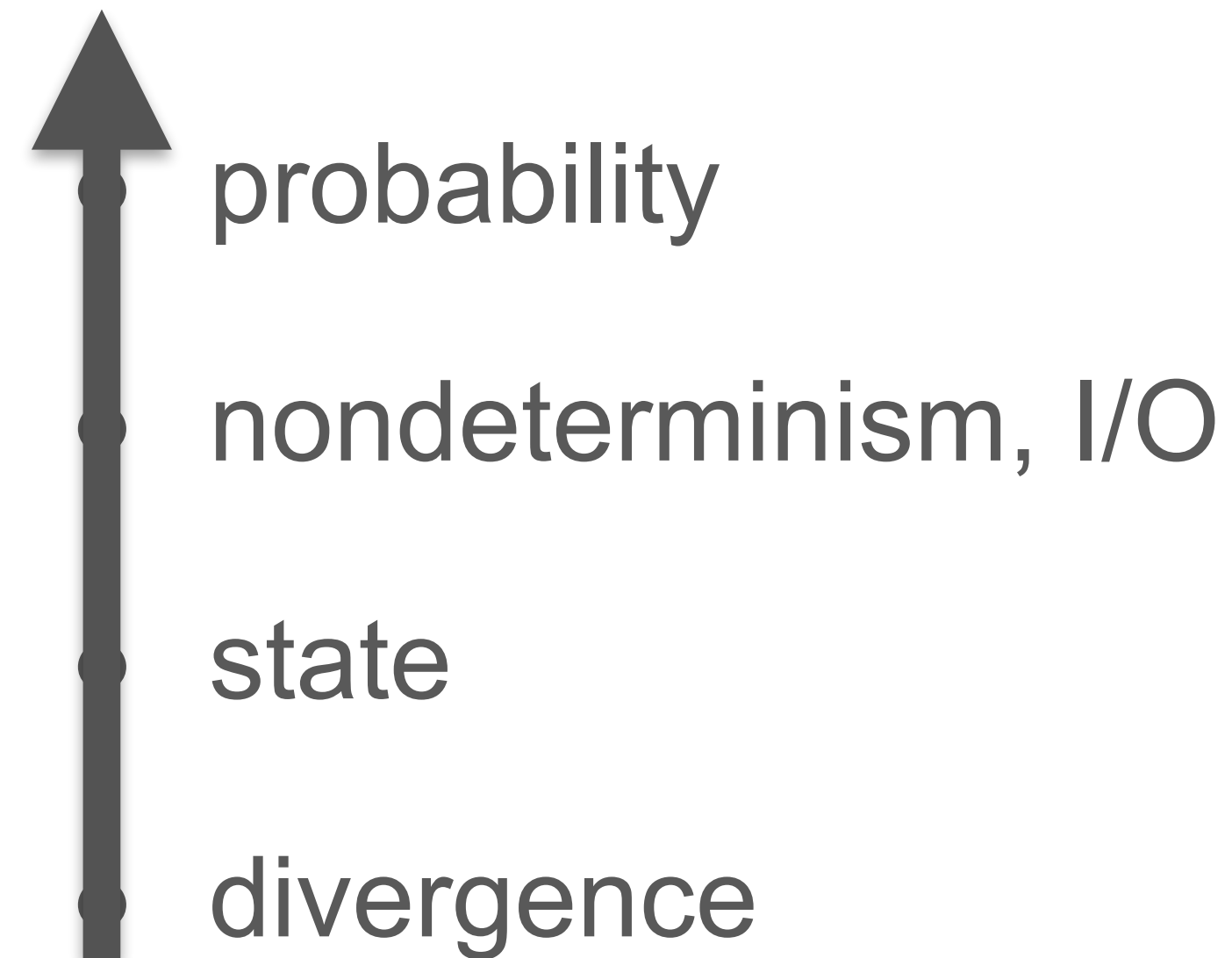
if $\exists R: PC\text{-simulation}$ such that $\forall C. (C[t], C[u]) \in R$

Extending the approach

	[M., PhD thesis '20] [M.+, arxiv '19]	[M.+, FLOPS '24]	[M.+, CMCS '24]
1. define a “simulation”	<i>counting simulation</i>	<i>local coherence</i> [Huet '80]	<i>preorder-constrained simulation</i>
2. fix a system	graph rewriting systems	term rewriting systems <i>term evaluation systems</i>	nondeterministic automata
3. prove a “simulation”	😞 elementary but tedious case analysis	😊 automatable <i>critical pair analysis</i>	(😞 game solving)
divergence	✓	✓	✓
continuation	(✓)	✓	(✓)
mutable state	✓	✗	(✓)
I/O	✗	✗	✓
angelic nondeterminism	✗	✗	✓
demonic nondeterminism	✗	✗	✗
probability	✗	✗	✗

Contextual improvement via “simulation”

Climbing up a ladder

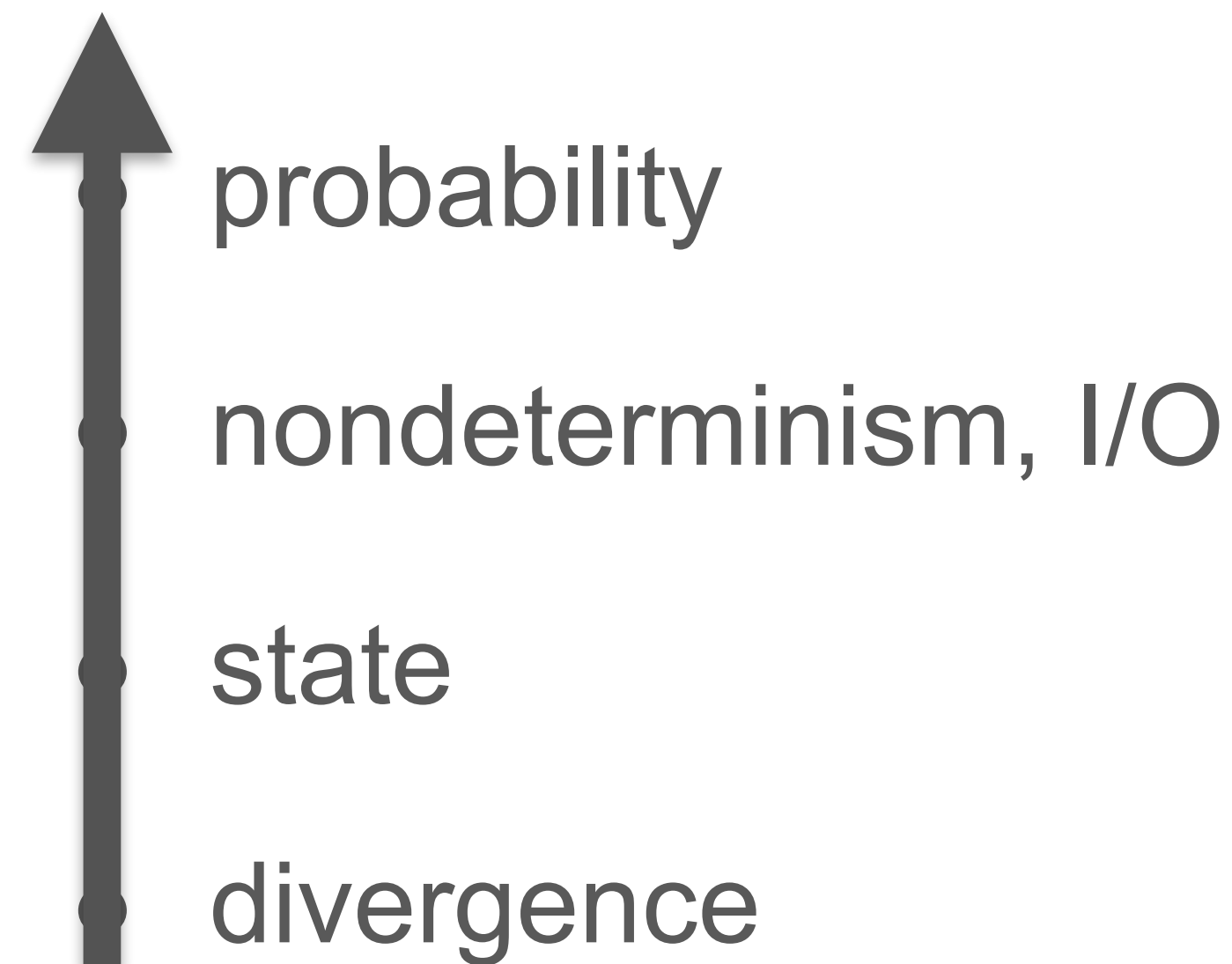


The left bar:
1. define a “simulation”

The right bar:
3. prove a “simulation”

Contextual improvement via “simulation”

Climbing up a ladder



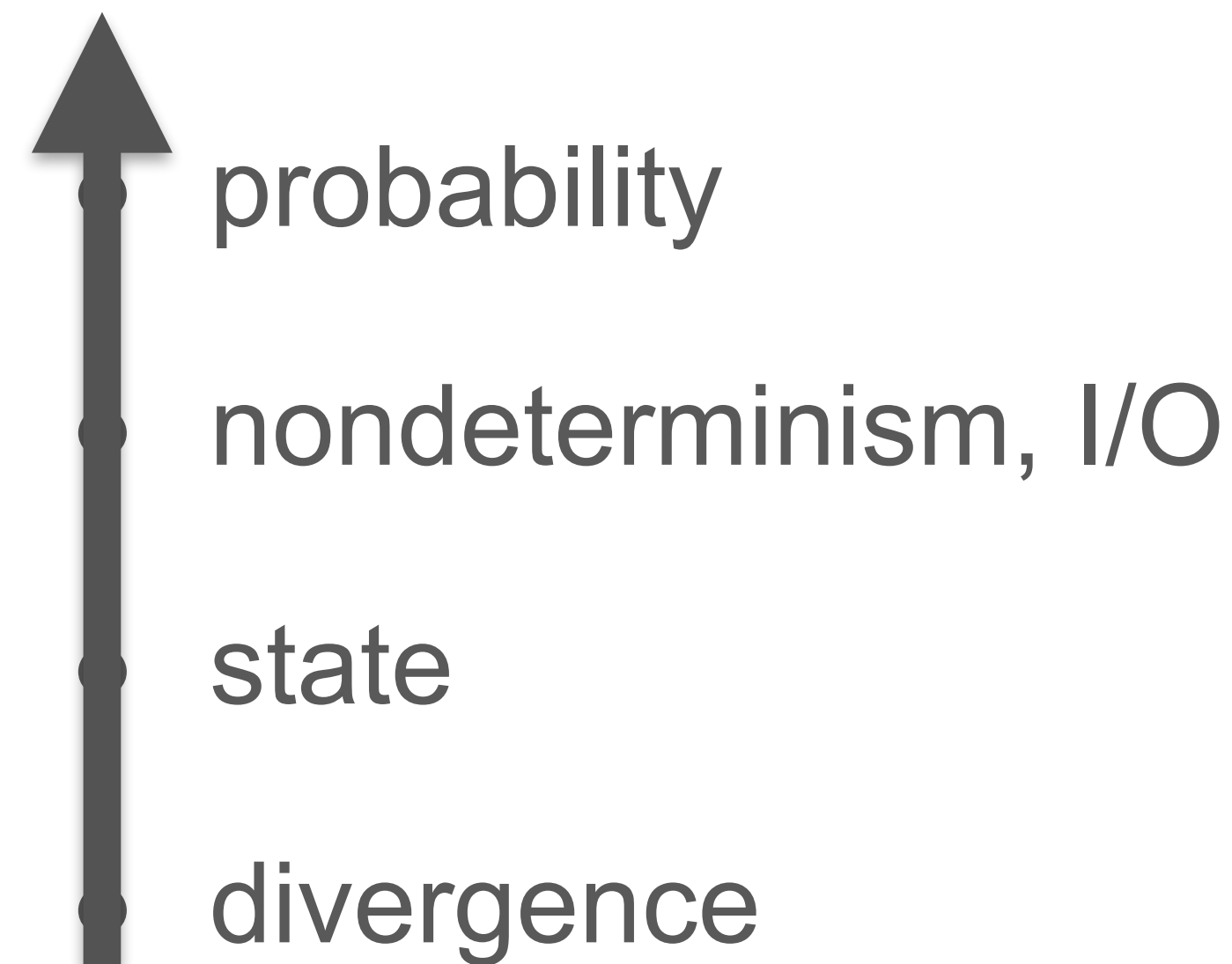
counting simulation & graphical local reasoning [M., PhD thesis '20]

The left bar:
1. define a “simulation”

The right bar:
3. prove a “simulation”

Contextual improvement via “simulation”

Climbing up a ladder



counting simulation & graphical local

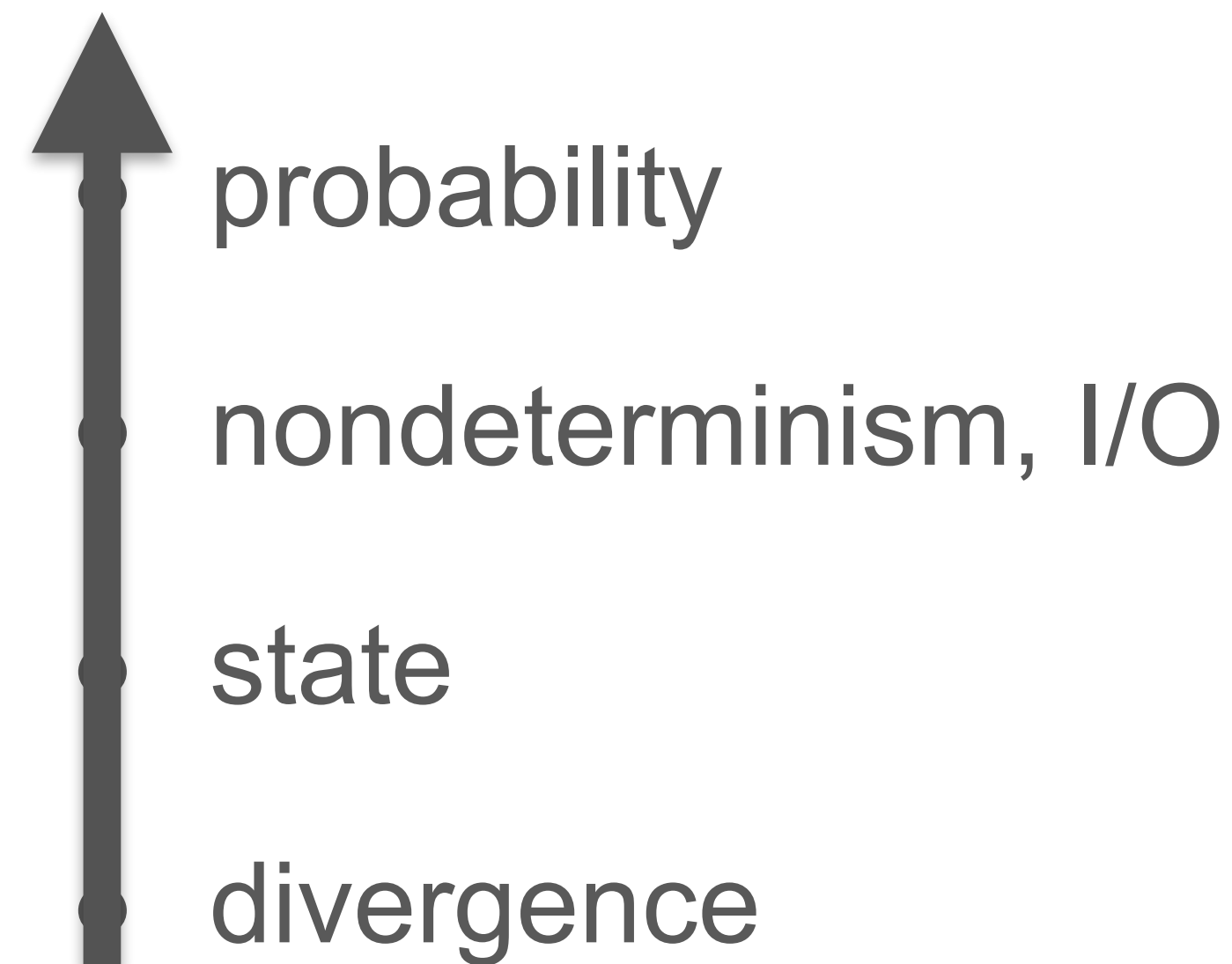
local coherence & critical pair analysis
[M.+, FLOPS '24]

The left bar:
1. define a “simulation”

The right bar:
3. prove a “simulation”

Contextual improvement via “simulation”

Climbing up a ladder



*preorder-constrained
simulation*
[M.+, CMCS '24]

counting simulation & graphical local

local coherence & critical pair analysis
[M.+, FLOPS '24]

The left bar:
1. define a “simulation”

The right bar:
3. prove a “simulation”

Contextual improvement via “simulation”

Proposed, refined & extended the approach

	[M., PhD thesis '20] [M.+, arxiv '19]	[M.+, FLOPS '24]	[M.+, CMCS '24]
1. define a “simulation”	<i>counting simulation</i>	<i>local coherence</i> [Huet '80]	<i>preorder-constrained simulation</i>
2. fix a system	graph rewriting systems	term rewriting systems <i>term evaluation systems</i>	nondeterministic automata
3. prove a “simulation”	😞 elementary but tedious case analysis	😊 automatable <i>critical pair analysis</i>	(😏 game solving)
divergence	✓	✓	✓
continuation	(✓)	✓	(✓)
mutable state	✓	✗	(✓)
I/O	✗	✗	✓
angelic nondeterminism	✗	✗	✓
demonic nondeterminism	✗	✗	✗
probability	✗	✗	✗